

OBSERVACIONES DE LA PRÁCTICA

Nicolás Fajardo Aguirre - 202413083
Christian Bravo - 202322012
Dennys Lozano - 202413859

Contents

Ambientes de pruebas 1

Máquina 1 2

Resultados para Queue con Array List..... 2

Resultados para Stack con Array List..... 2

Resultados para Queue con Linked List..... 2

Resultados para Stack con Linked List 3

Máquina 2 3

Resultados para Queue con Array List..... 3

Resultados para Stack con Array List..... 3

Resultados para Queue con Linked List..... 4

Resultados para Stack con Linked List 4

Máquina 3 4

Resultados para Queue con Array List..... 4

Resultados para Stack con Array List..... 5

Resultados para Queue con Linked List..... 5

Resultados para Stack con Linked List 5

Preguntas de análisis 6

Ambientes de pruebas

	Nicolas Fajardo	Dennys Lozano	Christian Bravo
Procesadores	AMD Ryzen 5 4600H with Radeon Graphics	12th Gen Intel(R) Core(TM) i5-12500H	AMD Ryzen 5 5600U
Memoria RAM (GB)	8 GB	16 GB	8 GB
Sistema Operativo	Windows 10	Windows 11	Windows 11

Tabla 1. Especificaciones de las máquinas para ejecutar las pruebas de rendimiento.

Máquina 1

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.080	0.095	0.004
5.00%	0.408	0.464	0.002
10.00%	0.661	1.095	0.001
20.00%	1.419	2.883	0.007
30.00%	1.968	3.608	0.002
50.00%	4.409	7.411	0.002
80.00%	5.503	18.440	0.002
100.00%	7.607	24.346	0.003

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Linked List)
0.50%	0.035	0.053	0.003
5.00%	0.347	0.452	0.001
10.00%	0.683	1.012	0.001
20.00%	2.487	1.999	0.031
30.00%	1.936	2.501	0.001
50.00%	4.615	4.311	0.002
80.00%	7.501	6.771	0.003
100.00%	7.438	9.875	0.002

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%	0.244	0.130	0.019
5.00%	0.918	1.234	0.002
10.00%	1.199	1.048	0.002
20.00%	3.376	2.210	0.002
30.00%	3.536	4.249	0.009
50.00%	6.514	7.585	0.003
80.00%	11.359	9.381	0.003
100.00%	19.836	13.144	0.003

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%	0.070	0.495	0.006
5.00%	0.689	7.103	0.003
10.00%	0.866	23.773	0.002
20.00%	2.783	100.866	0.003
30.00%	2.956	260.109	0.003
50.00%	6.386	744.605	0.005
80.00%	9.516	1814.438	0.004
100.00%	28.190	2836.770	0.005

Máquina 2

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.039	0.038	0.002
5.00%	0.203	0.364	0.006
10.00%	0.545	1.065	0.002
20.00%	0.822	1.608	0.001
30.00%	0.704	4.188	0.001
50.00%	3.061	20.110	0.001
80.00%	2.923	27.172	0.002
100.00%	7.867	62.841	0.003

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Array List)
0.50%	0.024	0.033	0.002
5.00%	0.286	0.422	0.002
10.00%	0.510	0.579	0.001
20.00%	1.378	2.113	0.004
30.00%	1.721	1.941	0.002
50.00%	2.269	3.439	0.002
80.00%	5.637	7.284	0.004
100.00%	4.654	5.457	0.002

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%	0.050	0.035	0.002
5.00%	0.287	0.299	0.001
10.00%	0.712	0.718	0.002
20.00%	1.187	1.437	0.002
30.00%	1.446	1.426	0.001
50.00%	2.027	2.674	0.001
80.00%	5.577	5.920	0.002
100.00%	6.721	7.478	0.002

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%	0.023	0.080	0.002
5.00%	0.281	7.726	0.001
10.00%	0.581	28.602	0.001
20.00%	1.121	84.531	0.002
30.00%	1.114	188.035	0.001
50.00%	2.721	709.854	0.001
80.00%	4.971	1786.293	0.012
100.00%	8.476	2618.903	0.004

Máquina 3

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.094	0.556	0.443
5.00%	0.342	0.589	0.002
10.00%	1.315	1.351	0.001
20.00%	2.198	0.899	0.001
30.00%	1.675	1.589	0.001
50.00%	3.646	3.295	0.001
80.00%	3.313	7.695	0.001
100.00%	5.303	10.626	0.001

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Array List)
0.50%	0.246	0.387	0.358
5.00%	0.445	0.232	0.002
10.00%	0.397	0.451	0.001
20.00%	0.871	1.014	0.001
30.00%	0.982	1.397	0.001
50.00%	2.920	2.408	0.002
80.00%	3.146	3.877	0.001
100.00%	4.689	5.250	0.001

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%	0.053	0.033	0.002
5.00%	0.387	0.286	0.002
10.00%	0.475	0.518	0.001
20.00%	1.046	1.030	0.001
30.00%	2.584	2.625	0.002
50.00%	2.635	2.641	0.001
80.00%	4.949	6.028	0.002
100.00%	5.410	6.166	0.002

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%	0.022	0.077	0.002
5.00%	0.203	3.528	0.003
10.00%	0.408	15.308	0.001
20.00%	0.998	64.661	0.001
30.00%	2.407	154.917	0.003
50.00%	3.576	401.641	0.001
80.00%	4.143	1014.482	0.002
100.00%	5.446	1598.543	0.002

Preguntas de análisis

1. ¿Se observan diferencias significativas entre las implementaciones con ArrayList y LinkedList para las funciones de Queue y Stack? ¿Cuál es más eficiente en cada operación? ¿Por qué una implementación es más rápida en ciertos casos?

Sí se notan diferencias bastante marcadas, y lo interesante es que no son iguales en las tres máquinas. Con la muestra completa (10.000 libros) en la máquina de Christian el enqueue en Array List fue de 7.607 ms contra 19.836 ms en Linked List, o sea casi el triple; en mi máquina la diferencia fue distinta: 7.867 ms vs apenas 6.721 ms, ahí casi se empatan, seguramente por la carga que tenía el pc en ese momento. En dequeue pasa lo contrario, Linked List gana en las tres pruebas, pero con brechas muy distintas: en la máquina de Christian gana por poco (24.346 ms vs 13.144 ms, más o menos 1.85 veces), en la de Dennys la diferencia es mucho más grande (62.841 ms vs 7.478 ms, casi 8.4 veces) y en la de Nicolás vuelve a ser más moderada (10.626 vs 6.166 ms). Push tiene el mismo comportamiento que enqueue y por la misma razón: Array List gana porque un append sale más barato que armar y enlazar un nodo nuevo. Donde sí hay una diferencia enorme y que se repite en las tres pruebas es en pop: Array List se mantiene entre 5 y 10 ms mientras que Linked List se dispara a más de 1500 ms, e incluso llega a 2836 ms en un caso. La razón es que nuestra Linked List no guarda un puntero al penúltimo nodo, entonces para hacer `remove_last` toca recorrer toda la lista cada vez, eso vuelve cada pop $O(n)$ y el ciclo completo termina siendo $O(n^2)$. Peek y top prácticamente no cambian entre implementaciones (todos por debajo de 0.01 ms), como se esperaría de operaciones $O(1)$.

2. ¿Cuándo es preferible usar ArrayList o LinkedList? Si insertamos y eliminamos con frecuencia, ¿qué estructura conviene más? Si accedemos aleatoriamente a elementos, ¿cuál es más eficiente?

Depende bastante de qué se vaya a hacer con la estructura. Si el patrón de uso es insertar y sacar por el frente todo el tiempo, como pasa en una cola, en teoría Linked List debería ganar siempre porque `add_last` y `remove_first` son $O(1)$ sin mover memoria, aunque en la práctica esa ventaja no siempre se nota tanto como uno esperaría (como se ve en el dequeue de la máquina de Christian, donde la diferencia es mínima). Si se necesita acceder a posiciones aleatorias Array List es mucho mejor, porque indexar es $O(1)$ contra recorrer nodo por nodo en la Linked List, que es $O(n)$. Y para una pila, donde las operaciones se concentran al final, Array List termina siendo la mejor opción con la implementación que hicimos, ya que `remove_last` en nuestra Linked List no quedó con puntero al penúltimo nodo y toca recorrer toda la lista, por lo que resulta más lento en vez de más rápido como uno pensaría solo viendo la teoría.

3. Durante la ejecución de las pruebas ¿Se presentan anomalías en los tiempos de ejecución que no se explican con la teoría?

La anomalía más rara la vimos comparando las tres máquinas entre sí, más que dentro de una sola prueba. En teoría dequeue con Array List debería ser siempre mucho más lento que con Linked List porque cada `remove_first` mueve todos los elementos restantes, pero la brecha que medimos cambia muchísimo según la máquina: en la de Christian fue de apenas 1.85 veces, en la de Dennys de casi 8.4 veces y en la de Nicolás volvió a bajar a 1.72 veces. No tenemos una explicación totalmente segura de por qué varía tanto, sospechamos que tiene que ver con que `list.pop(0)` en Python está hecho en C sobre

memoria contigua, entonces su costo real depende del hardware y de qué tan ocupado estuviera el procesador en el momento de correr la prueba (no cerramos todos los programas de fondo antes de medir). Otra cosa que llamó la atención es que en el pop de la pila con Linked List los tiempos se dispararon distinto en cada máquina (2836 ms en la de Christian, 2618 ms en la de Dennys y 1598 ms en la de Nicolás) aunque las tres corrieron el mismo código con el mismo tamaño de muestra, lo que sugiere que el costo de recorrer nodos pesa bastante y no es igual de un computador a otro.

4. Complete la siguiente tabla de acuerdo con qué operación es más eficiente en cada implementación (marque con una x la que es más eficiente). Adicionalmente, explique si este comportamiento es acorde con lo enunciado en la teoría. Justifique las respuestas.

		Array List	Linked List	Justificación
QUEUE	Enqueue()	X		Array List gana porque un append es básicamente pegar al final de un arreglo en memoria contigua, mientras que Linked List arma un nodo nuevo y mueve el puntero last cada vez, eso pesa un poco mas aunque las dos sean $O(1)$.
	Dequeue()		X	Aqui Linked List es mejor, tal como dice la teoría (mueve solo el puntero first en vez de correr todo el arreglo), pero la ventaja medida no fue igual de grande en las tres maquinas: chiquita en la de Christian y bastante mas notoria en la mía, así que puede que también influya la carga de cada pc en el momento de la prueba.
	Peek()	X	X	Empatados, los dos son $O(1)$ (uno indexa el arreglo, el otro sigue el puntero first) y se nota en las mediciones: todos los tiempos quedaron por debajo de 0.01 ms sin importar el tamaño de la muestra.
STACK	Push()	X		Mismo caso que enqueue: un append sale más barato que crear y enlazar un nodo nuevo.
	Pop()	X		Es la diferencia más grande de todo el laboratorio. Array List se mantuvo siempre entre 5 y 10 ms pero Linked List se disparó a más de 1500 ms (hasta 2836 ms en una de las maquinas), porque remove_last en nuestra implementación no guarda puntero al penúltimo nodo y toca recorrer toda la lista para sacar el ultimo elemento; el pop deja de ser $O(1)$ y pasa a ser $O(n)$, y el ciclo de prueba completo termina en $O(n^2)$.
	Top()	X	X	También empatados: las dos estructuras mantienen un puntero directo al último elemento, así que consultar el tope es $O(1)$ en ambas, y se nota en que los tiempos son bajitos y parecidos en las tres maquinas.

